

Hosting & Deployment Plan

iSCARB Platform | Prepared for the Backend Developer

Google Cloud (Dammam, in-Kingdom) with NVIDIA hosted AI | PDPL-aware

[CODE] = derived from the codebase, firm. [VERIFY] = depends on current provider offerings / pricing / compliance — confirm before committing.

1. The Plan in One Paragraph

Host the entire application stack — Next.js app, PostgreSQL (with pgvector), Redis, and object storage — inside **Google Cloud's Dammam (Saudi Arabia) region**, so all stored personal data stays in the Kingdom. The application must run on a **long-lived compute instance (VM or container), NOT serverless**, because lecture generation runs in-process for minutes and serverless platforms will kill it. **NVIDIA AI inference remains the hosted API** (integrate.api.nvidia.com) for now — a deliberate, approved decision for the pilot. This means AI prompts (student answers, faculty source excerpts) leave the Kingdom to reach NVIDIA; that cross-border transfer must be covered by a PDPL legal basis (compliance to confirm). A clear, low-effort path to move AI in-Kingdom later (self-hosted NIM) is documented in Section 6.

2. Why These Constraints (From the Code)

- [CODE] **Long-running processes required.** Lecture generation runs in-process (~minutes) and AI calls take ~40s each. Serverless will time out and kill them. Use a VM/container with a long-lived Node process (PM2 or Docker).
- [CODE] **PostgreSQL + pgvector.** Schema/migrations require Postgres with the pgvector extension.
- [CODE] **Redis.** Used for caching/queue work; needs an in-region instance.
- [CODE] **Object storage.** Faculty files/exports write to S3-compatible storage (src/lib/lecture/storage.ts, region env currently me-central-1). Needs in-Kingdom storage with the region set correctly.
- [CODE] **Long AI timeouts.** ~40s+ per call; AI-bound routes need generous timeouts and adequate concurrency for ~100 users.
- [CODE] **Migrations at deploy.** Run `prisma migrate deploy` as a deploy step — not a boot-time step.

3. Target Architecture (Dammam, In-Kingdom)

Layer	What to deploy	Notes
App runtime	Next.js on a Compute VM/container in Google Dammam [VERIFY]	Long-lived process (PM2/Docker). NOT serverless for generation.
Database	PostgreSQL + pgvector, in-region	Managed (Cloud SQL) [VERIFY pgvector] or self-managed. In Dammam.
Cache/queue	Redis, in-region	Managed (Memorystore) [VERIFY] or self-managed. In Dammam.
Object storage	In-Kingdom bucket	Set storage region env to the in-Kingdom bucket; confirm S3 client target.
AI inference	NVIDIA hosted API — unchanged for now	Cross-border transfer of prompt content. PDPL basis [VERIFY]. Migration path in §6.
Email/push/errors	SMTP, Web Push, Sentry/OTLP	Confirm each provider's region and payload (see §5).

Residency result: app, database, Redis, and file storage are all in-Kingdom. The ONLY personal data that leaves the Kingdom is the content of AI prompts sent to NVIDIA (student answer text, faculty source excerpts). No structured identifiers (name/email/national ID/student ID) are placed in those prompts by the code — but there is currently no PII-stripping step (see §5).

4. Deployment Checklist (Execute in Order)

1. **[VERIFY]** Provision Dammam resources: compute VM/container, in-region Postgres (pgvector-capable), in-region Redis, in-region object storage. Confirm each is physically in the Saudi region.
2. **Environment variables** — set all secrets in the deploy environment (not committed files): DATABASE_URL, REDIS_URL, NVIDIA_API_KEY..._5, JWT keys, PASSWORD_RESET_SECRET (≥32 chars), IMPERSONATION_JWT_SECRET, ISCARB_AUTH_DISABLED=false, storage creds + in-Kingdom region, SMTP creds. Confirm the full list against src/env.mjs.
3. **Secrets hygiene** — ensure no real secrets remain in .env.example or any committed file; rotate any ever committed.
4. **Database** — run prisma migrate deploy as a deploy step; confirm all migrations apply and migrate diff = zero drift on the fresh prod DB.
5. **Storage region** — set object-storage region/bucket env to the in-Kingdom bucket; verify a write lands in-Kingdom.
6. **Long-running process** — run under PM2/Docker so generation survives; confirm a full lecture generation completes without being killed.
7. **Timeouts/concurrency** — allow long request durations for AI routes (≥ ~40s+) and adequate concurrency for ~100 users.
8. **Build** — confirm next build completes for production.
9. **Smoke test in-region** — app boots, DB + Redis reachable, a real AI call succeeds, file upload/download works, a student can start the exam, a faculty lecture generates.
10. **[VERIFY]** Confirm non-NVIDIA egress regions (§5) are acceptable or relocated in-Kingdom.

5. Data-Residency / PDPL Summary

What leaves the Kingdom under this plan

- **AI prompt content to NVIDIA hosted API** — student answer text, faculty source excerpts, rubric/scenario/course content. **[CODE]** No structured identifiers are placed in prompts. **[CODE]** No PII-stripping step exists, so an identifier a student types inside their answer would be transmitted unchanged. NVIDIA's physical processing region is unknown from the code — treat as a cross-border transfer. **[VERIFY]** NVIDIA DPA, subprocessors, processing/log regions; confirm a PDPL basis permits sending answer text/source excerpts abroad for the pilot.

Recommended code hardening (worth doing on either path)

- Add a **PII-stripping / sanitization step** before any prompt is sent to an external AI (none exists today). Scrubs identifiers a student might embed in free text and materially reduces PDPL exposure. (See the companion Prompts document.)

Other external egress to confirm regions for **[VERIFY]**

- Object storage (S3-compatible) — faculty files/exports; region env currently me-central-1; confirm the actual bucket is in-Kingdom.
- SMTP email — student email + notification content.
- Web Push — push endpoint + payload to browser push infrastructure.
- Sentry (if SENTRY_DSN) — exceptions + optional context; confirm region or disable in prod.
- OpenTelemetry OTLP (if configured) — traces to the collector URL; confirm region.
- Wikimedia/Wikipedia APIs — topic/search queries (not student PII by design).
- Postgres / Redis — full personal-data store; residency depends on hosting → this plan places them in-Kingdom.

Bottom line: with this plan everything is in-Kingdom **except** AI inference (NVIDIA) and any third-party service above not yet confirmed in-region. The AI cross-border transfer is the main item requiring a documented PDPL basis.

6. Migration Path to Fully In-Kingdom AI (Later — Not Needed Now)

If/when the cross-border AI transfer must be eliminated, moving inference in-Kingdom is a **contained change**, **not a rewrite** (confirmed from the code):

- **[CODE]** Almost all AI traffic flows through chatJson / chatJsonRaw in src/lib/ai-engine.ts, where the URL `https://integrate.api.nvidia.com/v1/chat/completions` is **hardcoded**; the primary path does not read `NVIDIA_BASE_URL` / `OPENAI_BASE_URL`.
- **Change needed:** make `getClient()/chatJson*` read `process.env.NVIDIA_BASE_URL || OPENAI_BASE_URL || default` (the secondary clients `llm-verifier.ts` and `nvidia-vlm-gate.ts` already do this), then point the env at a self-hosted NIM on an in-Kingdom GPU.
- **Also:** parameterize `generateImage()`'s hardcoded image URL if used; swap Bearer auth to what the NIM expects; align model slugs to the NIM's catalog; validate VLM `json_object` mode on the NIM.
- **Effort:** localized to `ai-engine.ts` + `env/ops` for the NIM + model-catalog alignment. Callers don't change (they go through the shared helpers).
- **Cost [VERIFY]:** self-hosting NIM needs an in-Kingdom GPU instance and, for production, NVIDIA AI Enterprise licensing — the main cost of full in-Kingdom AI.

7. Suggestions — Faster AI Models

The platform's main latency is the AI calls themselves (~40s+ on the current model path). Hosting location barely changes this — model compute time dominates. Options below are all **[VERIFY]** — confirm current availability, latency, cost, and PDPL fit before adopting.

A. Smaller/faster model for simpler calls

- Much AI work (scoring a fixed MCQ answer, short reviewer passes) is simpler than essay grading and may not need the largest model. A smaller/fast-inference model can respond in seconds. Match model size to task.
- **Exam note:** the product direction is **keyed MCQ scoring** (compare selected option to the stored correct answer) — this removes the AI call from scoring entirely and makes scoring instant. It is the biggest exam-latency win and should be prioritized over model choice for the exam path.

B. Parallelize with bounded concurrency

- Where multiple AI calls happen per request (batch scoring, multi-pass review), run them concurrently within a safe concurrency limit rather than sequentially. Tune to what the provider tolerates without rate-limiting. Cuts wall-clock time without changing quality.

C. Move generation off the user's waiting path

- Prepare content ahead of time (e.g. exam questions generated up front in the background) so the user never waits through live generation. Combined with keyed scoring, the candidate experiences near-zero AI wait.

D. Self-hosted NIM on a fast in-Kingdom GPU (dual benefit)

- A dedicated modern GPU (H100/A100/L40S-class) running the model locally can outperform a shared hosted endpoint AND keeps data in-Kingdom. Trade-off is GPU + licensing cost. Model once volume grows.

E. Alternative fast-inference providers/models [VERIFY + PDPL]

- Faster-inference providers and newer efficient open models exist. ANY option must be checked against PDPL data-residency — a faster model that ships data to a non-compliant region is not acceptable. Evaluate on three axes together: latency, quality for THIS task, and data-residency.

Recommended priority for speed: (1) keyed MCQ scoring [removes AI from scoring] → (2) up-front/background generation [removes waiting] → (3) parallelism with tuned concurrency → (4) smaller model

for simple calls → (5) self-hosted NIM on a fast GPU when scaling / for full compliance.

8. Decisions That Need a Human (Not the Dev's Call Alone)

- **[VERIFY — compliance]** Is sending anonymized-but-personal answer text and faculty source excerpts to NVIDIA's hosted API (outside the Kingdom) permissible under PDPL for the pilot? This decides whether NVIDIA can stay hosted or must move in-Kingdom now.
- **[VERIFY — institution]** Does the institution require a Saudi-owned/sovereign provider, or does an in-Kingdom hyperscaler region (Google Dammam) satisfy them?
- **[VERIFY — provider]** Confirm Google Dammam region availability, pgvector support on the managed DB, and current pricing at ~100-user scale before committing.

This document is a plan and summary for implementation. Requirements marked [CODE] are grounded in the codebase; items marked [VERIFY] must be confirmed against current provider documentation and a compliance review before go-live.